

Grammar-Derived Operation Abstraction for Cross-Project Vulnerability Detection

Anonymous Authors

Abstract

Deep learning detectors of software vulnerabilities generalise poorly across projects, and the field has responded largely by proposing new architectures. We ask a different question: how much of the cross-project gap is attributable to the *representation* rather than the model? We specify ϕ , a total labelling function from the node alphabet of a C grammar onto a small operation taxonomy, together with a projection lattice that makes its three granularities nested by construction. Because the resulting alphabet is fixed by the grammar rather than by any project’s vocabulary, the representation is identical across corpora by construction. We evaluate ϕ twice. As a static analysis, we annotate a stratified sample of call sites and report per-class precision and recall, which exposes failure modes that an accuracy-only evaluation hides — notably that substring-based recognition of wrapped allocators misclassifies writes and logging macros as memory operations. As a representation, we hold graph topology, backbone, depth, readout and training budget fixed and vary only the node features, under project-level GroupKFold with deduplication, mega-projects held train-only, and cluster-bootstrap confidence intervals over held-out projects. Replacing lexical node features with the abstraction improves cross-project ranking by +0.064 AUC (95% CI [+0.028, +0.112], $p = 0.001$) and +0.022 AUPRC (95% CI [+0.000, +0.049], $p = 0.047$) on BigVul, winning 8 of 10 folds. We report negative results with equal weight: learned edge weighting and affinity-weighted federated aggregation do not help, and refining the taxonomy beyond eight types does not either. All code, annotations and the scripts that regenerate every number are released.

Keywords: Program Representation, Static Analysis, Abstract Syntax Trees, Software Vulnerability Detection, Federated Learning, Graph Neural Networks, Differential Privacy

1. Introduction

Automated vulnerability detection has become a proving ground for graph learning: a function is parsed into a graph, a neural network is trained over it, and detection accuracy is reported. Progress is usually claimed architecturally — a different message-passing scheme, an attention variant, a language-model hybrid. Yet replication studies keep finding the same thing: whatever the architecture, performance collapses when the model is evaluated on projects it has never seen [1, 2, 3]. Detectors that appear strong within a project transfer poorly across projects, and successive architectures have not closed that gap.

This paper examines a factor that most of that work holds constant: what a node in the graph actually *carries*. Nearly all graph-based detectors embed the node’s lexical content — an identifier, an API name, a literal. Such a vocabulary is a property of a codebase, not of a language. FFmpeg allocates with `av_malloc`, GLib with `g_malloc`, the Linux kernel with `kmalloc`, OpenSSL with `OPENSSL_malloc`. A model whose features are drawn from one project’s vocabulary has, at test time on another project, largely out-of-vocabulary input. The mechanism the model must learn — an allocation whose size is computed, an indexed write, a missing bound — is shared; the tokens denoting it are not.

We therefore study a program abstraction rather than an architecture. We define ϕ , a total labelling function from the node-kind alphabet of the tree-sitter C grammar onto an operation taxonomy of eight types, with finer 16- and 32-type refinements related by explicit surjections so that the three granularities are nested by construction (Proposition 1). The resulting node alphabet is fixed by the grammar and the taxonomy, so it is identical for every project and every corpus — the property we claim is responsible for transfer.

Two evaluations follow, and we regard the first as being as important as the

second. Abstractions of this kind are usually introduced as preprocessing and never validated: no precision, no recall, no error analysis. We instead evaluate ϕ as a static analysis, annotating a stratified sample of call sites and reporting per-class precision and recall. That exercise is not ceremonial. It found that substring-based recognition of wrapped allocators labels `__put_user` and `skb_put` — both writes — as release operations, treats a floating-point predicate as a block set, and misses `copy_to_user`; and that classifying `printf` and `fprintf` as buffer-writing formatters silently counts 655 stream-I/O call sites in one corpus as memory operations. Both defects are invisible to downstream accuracy and were corrected before the experiments reported here.

The second evaluation isolates the representation. Graph topology, backbone, depth, hidden width, readout, optimiser and gradient budget are held fixed; only the node features vary. Evaluation uses project-level GroupKFold with near-duplicate removal before splitting, mega-projects held train-only so that no “cross-project” fold is secretly a single repository, vocabularies fitted on training projects only, and cluster-bootstrap confidence intervals that resample held-out *projects* rather than functions.

Our contributions are:

- **A specified abstraction.** ϕ as a total, deterministic labelling over a grammar alphabet, with a projection lattice and a nesting proposition, requiring no name resolution, build system or whole-program context (Section 3).
- **Validation as an analysis.** Per-class precision and recall against annotated ground truth, with the measured precision/coverage trade-off between exact allow-listing and component-based recognition of project-wrapped APIs, and the failure modes each admits (Section 4.2).
- **The representation effect, measured under controls.** Replacing lexical features with the abstraction improves cross-project ranking by +0.064 AUC ($p = 0.001$) and +0.022 AUPRC ($p = 0.047$), winning 8 of 10 folds, with every other factor held constant (Section 4.3).

• **Negative results, reported with equal weight.** A learned edge-weighting module and an affinity-weighted federated aggregation scheme — both of which we implemented and initially believed in — do not improve on their ablations; nor does refining the taxonomy beyond eight types (Section 4.5).

Section 2 positions the work, Section 3 specifies the abstraction, Section 4 reports both evaluations, and Section 5 discusses what the result does and does not license.

2. Background and Related Work

2.1. Deep Learning for Software Vulnerability Detection

The application of Deep Learning (DL) to Software Vulnerability Detection (SVD) has evolved significantly from early sequence-based models to modern graph-based and transformer-based architectures. Early works, such as VulDeePecker [4] and SySeVR [5], treated code as sequences of tokens, employing recurrent neural networks (RNNs) like BiLSTMs to classify code gadgets. While these sequence-based models demonstrated the feasibility of DL for SVD, they struggled to capture the complex, non-linear structural dependencies inherent in source code.

The introduction of the Code Property Graph (CPG) [6], which unifies the Abstract Syntax Tree (AST), Control Flow Graph (CFG), and Data Flow Graph (DFG), marked a paradigm shift. Graph Neural Networks (GNNs) became the architecture of choice for SVD because they naturally align with the topological structure of CPGs. Devign [7] pioneered this approach by applying Gated Graph Neural Networks (GGNNs) over composite graphs to detect vulnerabilities in large C/C++ projects. Subsequent GNN-based models have focused on improving graph representation and interpretability. For instance, DeepWukong [8] operates on inter-procedural program slices, while IVDetect [9] provides fine-grained interpretations using GNNExplainer. More recent advancements

85 include Vul-LMGNNs [10], which fuses Large Language Models (LLMs) with GNNs, and HAGNN [11], which utilises heterogeneous attention mechanisms.

Simultaneously, Transformer-based models and LLMs have been adapted for SVD. LineVul [12] leverages CodeBERT for line-level vulnerability prediction, and LLMxCPG [13] uses CPGs to guide LLM attention. Despite these advances,
90 recent studies [14, 15] indicate that LLMs often operate at a shallow structural level, highlighting the continued necessity for deep structural abstractions. Furthermore, rigorous replication studies [1, 2, 3] have consistently exposed a severe generalisation gap: SOTA models perform well in within-project settings but their accuracy drops precipitously when evaluated on unseen projects (cross-
95 project).

2.2. Cross-Project Vulnerability and Defect Prediction

Cross-Project Vulnerability Detection (CPVD) and Cross-Project Defect Prediction (CPDP) aim to train models on a source project and deploy them on a target project. Traditional CPDP often relies on software metrics and transfer
100 learning [16]. In the context of DLVD, several works have employed Domain Adaptation (DA) techniques. CD-VulD [17] and VulGDA [18] use deep domain adaptation on graph embeddings to achieve cross-domain discovery. CPVD [19] combines Graph Attention Networks (GATs) with adversarial domain adaptation, representing the current SOTA for fully supervised cross-project SVD.
105 CSVD-TF [20] fuses expert and semantic metrics using TrAdaBoost. However, a critical limitation of these DA-based approaches is that they require centralised access to raw source code from both the source and target domains, violating privacy constraints in many real-world scenarios.

2.3. Federated Learning in Software Engineering

110 Federated Learning (FL) [21] allows multiple clients to jointly train a global model without sharing their local data. Its application to Software Engineering (FL-SE) is a burgeoning area of research, motivated by the proprietary nature

of commercial codebases [22]. Recent empirical studies [23, 24] confirm the feasibility of FL for tasks like code clone detection and vulnerability detection.

115 However, adapting FL to SVD presents unique challenges, primarily due to severe non-IID data distributions (heterogeneity in CWE types and coding styles). FedGraphNN [25] demonstrated that standard federated GNNs often underperform centralised models on non-IID graph splits. Existing FL-SVD
120 solutions typically rely on NLP models (e.g., CodeBERT) or naive GNN aggregations [26, 24], which inherit the generalisation gap of their centralized counterparts. To address graph heterogeneity, approaches like FedDense [27] decouple structural and feature learning. VulMorph-Fed builds upon these concepts but specifically targets the structural invariance of vulnerabilities by federating semantic prototypes rather than raw model parameters.

125 2.4. Privacy-Preserving Mechanisms

Ensuring data privacy in FL often involves Differential Privacy (DP) [28] or Secure Aggregation [29]. DP algorithms, such as DP-SGD, add calibrated noise (e.g., Laplace or Gaussian) to model updates. The magnitude of this noise is proportional to the sensitivity of the function being computed. Standard DP-SGD
130 on large GNNs or LLMs requires significant noise due to the high dimensionality of the model parameters and the large vocabulary size of source code tokens, severely degrading utility. VulMorph-Fed mitigates this by applying DP at the level of semantic prototypes. By utilizing a compact, abstracted morphological space instead of the full token vocabulary, the sensitivity is drastically reduced,
135 enabling a much more favorable privacy-utility trade-off.

3. Operation Abstraction

The object of study is a labelling function that replaces a program’s project-specific lexical surface with a small alphabet of operation types fixed by the language grammar. This section specifies it, states the projection lattice relating
140 its three granularities, and defines the graph on which it operates. Section 4

then evaluates it twice: as a static analysis, against annotated ground truth, and as a representation, by its effect on cross-project detection.

3.1. Program Graphs

Each function is parsed with `tree-sitter`, an incremental, error-tolerant
 145 parser that recovers a usable tree from non-preprocessed C. That property mat-
 ters here: vulnerability corpora ship isolated functions without their headers,
 so a parser requiring a complete translation unit cannot be used. From the
 resulting tree we build $G = (\mathcal{V}, \mathcal{E})$:

- $\mathcal{V}$: named AST nodes in pre-order, capped at 200;
- 150 • $\mathcal{E}_{AST}$: parent-child edges, both directions;
- $\mathcal{E}_{SIB}$: consecutive named siblings, giving source order;
- $\mathcal{E}_{DU}$: a def-use approximation linking successive occurrences of the same
 identifier leaf.

with $\mathcal{E} = \mathcal{E}_{AST} \cup \mathcal{E}_{SIB} \cup \mathcal{E}_{DU}$, giving on average ~ 130 nodes and ~ 350
 155 edges per function. We state plainly that this is a syntax graph with a data-
 dependence approximation, not a Code Property Graph [6]: interprocedural
 control flow and points-to-precise dependence are absent. Section 5.4 discusses
 what that bounds.

3.2. The Labelling Function ϕ

160 Let K be the node-kind alphabet of the tree-sitter C grammar ($|K| = 363$)
 and $\mathcal{T}_{32}$ the operation alphabet of Table 1. The abstraction is a total function

$$\phi : \mathcal{V} \longrightarrow \mathcal{T}_{32} \cup \{\perp\}, \quad (1)$$

where $\perp$ (UNKNOWN) labels nodes carrying no operation semantics. ϕ is deter-
 ministic and needs no name resolution, no build system and no whole-program
 context. It is defined by cases on the node’s grammar kind:

- 165 1. **Call expressions** are labelled by their callee name through ψ (Section 3.2.1); an unrecognised callee yields `CALL_SITE`. User-defined functions, macros and calls through function pointers therefore all receive a label without resolution.
2. **Kind-determined nodes** follow a fixed table: `subscript_expression`
 170 $\rightarrow$ `ARRAY_INDEX`; `field_expression` $\rightarrow$ `FIELD_ACCESS`; `cast_expression`
 $\rightarrow$ `CAST`; `if/switch/case` $\rightarrow$ `BRANCH_*`; `loops` $\rightarrow$ `LOOP_*`; `break/continue/goto/return`
 $\rightarrow$ `JUMP_*/RETURN`; `assignment_expression` and initialising declarators
 $\rightarrow$ `ASSIGN`; `update_expression` $\rightarrow$ `ARITH_ADD`.
3. **Operator-determined nodes**: a `binary_expression` is labelled by its
 175 operator token (`+-` $\rightarrow$ `ARITH_ADD`; `*/` $\rightarrow$ `ARITH_MUL`; `%` $\rightarrow$ `ARITH_MOD`;
`bitwise` $\rightarrow$ `BITWISE_OP`; `shifts` $\rightarrow$ `SHIFT_OP`; `equality` $\rightarrow$ `COMPARISON_EQ`;
`relational` $\rightarrow$ `COMPARISON_REL`; `logical` $\rightarrow$ `LOGICAL_OP`). A `pointer_expression`
 is `PTR_DEREF` or `ADDR_OF` by its prefix operator; unary `!` is `LOGICAL_OP`.
4. **Otherwise** $\phi(v) = \perp$.

180 3.2.1. Callee classification ψ

ψ maps a callee name to an operation class in two stages.

Stage 1 (exact). A fixed allow-list of C standard-library names, grouped by the memory effect they carry: allocation, reallocation, release, block copy, block set, string copy, concatenation, buffer-writing formatting, length query,
 185 and stream I/O.

Stage 2 (component). Production C rarely calls those directly: FFmpeg wraps allocation as `av_malloc`, GLib as `g_malloc`, the Linux kernel as `kmalloc`, OpenSSL as `OPENSSL_malloc`. Stage 2 splits the callee identifier into components on underscore and camel-case boundaries and fires when a *whole*
 190 *component* matches an operation word.

Whole-component matching, rather than substring matching, is load-bearing. Substring matching — the obvious implementation — is measurably wrong on real code: it labels `__put_user` and `skb_put` (writes) as release operations via “put”, `gen_new_label` as allocation via “new”, `float64_is_zero` (a predicate)

195 as a block set via “zero”, and `reallocate_view` as reallocation, while *missing*
`copy_to_user`, a genuine bounded copy. Section 4.2 quantifies the resulting
error.

We further separate stream I/O from buffer formatting. `printf` and `fprintf`
write to a stream and carry no caller-buffer overflow semantics; only the `s*printf`
200 and `scanf` families fill a caller-supplied buffer. The distinction is not cosmetic:
in our Devign sample `fprintf` and `printf` alone account for 655 call sites, every
one of which a conflated rule would count as a memory operation.

3.3. The Projection Lattice

Three granularities are defined — $\mathcal{T}_{32}$ (finest), $\mathcal{T}_{16}$, $\mathcal{T}_8$ — related by surjec-
205 tions

$$\pi_{16} : \mathcal{T}_{32} \rightarrow \mathcal{T}_{16}, \quad \pi_8 : \mathcal{T}_{16} \rightarrow \mathcal{T}_8, \quad (2)$$

each extended to fix $\perp$. Coarser labellings are compositions, $\phi_{16} = \pi_{16} \circ \phi$ and
 $\phi_8 = \pi_8 \circ \pi_{16} \circ \phi$, so the granularities are nested *by construction* rather than
separately specified.

Proposition 1 (Nesting). *For all $u, v \in \mathcal{V}$, $\phi(u) = \phi(v) \Rightarrow \phi_{16}(u) = \phi_{16}(v) \Rightarrow$
210 $\phi_8(u) = \phi_8(v)$. Equivalently, the partition of $\mathcal{V}$ induced by ϕ_8 is coarser than
that induced by ϕ_{16} , which is coarser than that induced by ϕ .*

Proof. Immediate from Eq. (2): ϕ_{16} and ϕ_8 are compositions of ϕ with total
functions, and applying a function to equal arguments yields equal results. The
induced partitions are the preimages of a chain of surjections, hence a chain of
215 coarsenings. $\square$

The consequence is that varying granularity varies exactly one quantity —
the coarseness of a single labelling — so the granularity experiment in Section 4
is interpretable, which a comparison of three independently designed schemes
would not be.

Table 1: The $|\mathcal{T}_8| = 8$ operation alphabet and the $\mathcal{T}_{32}$ classes each abstracts. $\perp$ (UNKNOWN) labels nodes with no operation semantics: declarations, identifiers, literals, punctuation.

$\mathcal{T}_8$	abstracts ($\mathcal{T}_{32}$)	carries
MEMORY_ACCESS	MEMORY_{ALLOC,REALLOC,FREE,COPY,SET}, STRING_{COPY,CONCAT,FORMAT,LENGTH}	buffer lifetime, bounds
ARRAY_INDEX	ARRAY_INDEX	indexed access
PTR_DEREF	PTR_DEREF, ADDR_OF, FIELD_ACCESS, CAST	indirection
CONTROL_BRANCH	BRANCH_{IF,SWITCH}, LOOP_{FOR,WHILE}, JUMP_{BREAK,GOTO}, RETURN	reachability
ARITH_OP	ARITH_{ADD,MUL,MOD}, BITWISE_OP, SHIFT_OP	value computation
COMPARISON	COMPARISON_{EQ,REL}, LOGICAL_OP	guards
CALL_SITE	CALL_SITE, IO_CALL	unresolved or non-memory calls
ASSIGN	ASSIGN	value flow

220 3.4. Node Features

A node is represented as the sum of two embeddings, both over *project-invariant* alphabets:

$$\mathbf{x}_v = E_{\text{op}}[\phi_8(v)] + E_{\text{kind}}[\kappa(v)], \quad (3)$$

where $\kappa(v) \in K$ is the grammar kind and $E_{\text{op}}, E_{\text{kind}}$ are learned tables with $|\mathcal{T}_8| + 1 = 9$ and $|K| + 1 = 364$ rows. Both alphabets are fixed by the C
 225 grammar and the taxonomy, hence identical across projects and corpora. That invariance is the property we claim explains transfer, and it is what Section 4 tests against a lexical baseline under an otherwise identical protocol.

One caveat, stated precisely. Node *features* contain no identifier, but ϕ *consults* identifier text to classify a callee, so a bounded amount of name-derived
 230 information — at most $\log_2(|\mathcal{T}_{32}| + 1) \approx 5$ bits per call node — influences the representation. What does not survive is the identifier itself, its spelling, or any project-specific vocabulary.

4. Evaluation

We evaluate the abstraction twice, and report negative results alongside
 235 positive ones:

- **RQ1 (as an analysis).** How accurately does ϕ label operations, per class, and what does component-based recognition of wrapped APIs buy and cost relative to exact allow-listing?

- **RQ2 (as a representation).** With graph, backbone and training budget held fixed, how much does replacing lexical node features with the abstraction change cross-project detection?
- **RQ3 (generality).** Does the effect hold across corpora and across message-passing schemes?
- **RQ4 (what does not work).** Do a learned edge-weighting module, an affinity-weighted federated aggregation, or a finer taxonomy improve on their ablations?

4.1. Setup

4.1.1. Corpora

We use BigVul [30], DiverseVul [31] and PrimeVul [3], loaded from pinned public mirrors. Each is sampled to 8,000 functions. We disclose two properties that are usually left implicit and that materially affect cross-project claims: the sample is taken in corpus order rather than at random, and project mass is highly concentrated — in BigVul, Chrome and `linux` together supply about two thirds of the functions.

4.1.2. Deduplication

Near-duplicate functions are removed *before* any split. Vended code appears under several project names, so without this step a held-out function can have a twin in training and project-disjointness is defeated silently. Signatures are computed over the abstracted node sequence, which catches renamed copies that text hashing misses; 2–3% of functions are removed.

4.1.3. Cross-project protocol

Evaluation is repeated project-level GroupKFold: projects are dealt into five folds, no project is split, and each fold is used once as the test set. Two provisions matter. First, projects too large to be placed without dominating a fold — Chrome and `linux` — are held *train-only* and reported as such, so that no fold labelled “cross-project” is in fact an evaluation on one repository. Second,

all vocabularies and label bucketings are fitted on training projects only. The resulting folds are stable: test fraction 0.071 on every fold, 39–49 test projects per fold.

270 We report AUC as the primary metric and AUPRC as the imbalanced-data companion, both threshold-free. F1 is reported only against an explicit floor: the trivial classifier that labels every function vulnerable achieves $F_1 = 0.161$ at the 8.8% prevalence of these test sets, which is above several F1 values reported as results in the literature.

275 4.1.4. Statistical treatment

Fold-level means are reported with standard deviations, but inference uses a *cluster bootstrap* that resamples held-out projects with replacement (1000 resamples), because functions within a repository share style, APIs and often near-duplicate code and are therefore not independent. We report the paired
290 difference between systems with a 95% percentile interval and a two-sided bootstrap p -value. Where an interval crosses zero we say so and draw no conclusion.

4.2. RQ1: ϕ as a static analysis

We sample call sites stratified over predicted classes, plus a frequency-weighted sample of unmatched names so recall is estimable, and annotate the true class
285 from documented API semantics.

Two findings are worth stating before any accuracy number. First, substring matching — the natural way to recognise wrapped allocators — is wrong in a patterned way: it labels `__put_user` and `skb_put` (writes) as release operations, `gen_new_label` as allocation, `float64_is_zero` (a predicate) as a
290 block set, and `reallocate_view` as reallocation, while missing `copy_to_user`. Whole-component matching removes these while retaining the genuine wrappers (`av_malloc`, `g_free`, `qemu_ram_alloc`, `av_freep`, `g_strdup`). Second, placing `printf` and `fprintf` among buffer-writing formatters causes 655 stream-I/O call sites in the Devign sample to be counted as memory operations; only the
295 `sprintf` and `scanf` families write into a caller-supplied buffer.

Table 2: RQ2: node-feature mode under an otherwise identical protocol (BigVul, 10 project-level folds, mean $\pm$ std). Differences are paired per fold; intervals are cluster bootstrap over held-out projects.

Node features	AUC	AUPRC
Trivial (all-positive)	0.500	0.088
Lexical token	0.587 ± 0.050	0.125 ± 0.040
Operation + grammar kind	0.668 ± 0.027	0.164 ± 0.034
Paired difference	+0.064	+0.022
95% CI (cluster bootstrap)	[+0.028, +0.112]	[+0.000, +0.049]
p (two-sided)	0.001	0.047
Folds won	8/10	8/10

Component matching raises coverage of call sites from 4.7% (exact allow-list) to 10.6%. We regard the residual disagreements as design information rather than noise, and name the hardest: `tcg_temp_free` frees an IR temporary in QEMU’s own allocator discipline rather than heap memory, and `g_strdup_printf` both allocates and formats, which a single-label taxonomy cannot express.

4.3. RQ2: the representation effect

Everything except the node features is held fixed: graph topology, backbone (GGNN), depth, hidden width, mean-max readout, classification head, optimiser, class weighting and gradient budget. Table 2 reports BigVul over ten folds.

The abstraction improves cross-project ranking by +0.064 AUC ($p = 0.001$) and +0.022 AUPRC ($p = 0.047$), winning 8 of 10 folds on both metrics, with both intervals excluding zero. It is also markedly more stable across folds (AUC standard deviation 0.027 versus 0.050), which is what one expects if the lexical model’s fold-to-fold variation is driven by how much of the test vocabulary it happened to see in training.

Two honest qualifications. Absolute performance remains modest: AUPRC of 0.164 against a base rate of 0.088 is a lift of roughly $1.9\times$, and cross-project
315 detection is not solved by this or, on the present evidence, by any published method. And the AUPRC interval’s lower bound is +0.000: the effect on precision at the top of the ranking is real but small, and we do not claim more.

4.4. RQ3: generality

The same contrast is run on a second corpus and with two further message-
320 passing schemes (GAT, GIN) to test whether the effect is specific to one dataset or one architecture. Results are reported in Table ??; the design is identical to RQ2 in every respect other than the factor named.

4.5. RQ4: what does not work

We report three negative results at the same length as the positive one,
325 because each corresponds to a mechanism we implemented and expected to help.

Learned edge weighting. A module that learns a soft importance mask over edges, trained jointly with the classifier and regularised toward sparsity, does not improve on its own ablation; removing it leaves AUC unchanged and
330 slightly improves F1.

Affinity-weighted aggregation. In a federated variant, weighting client contributions by the cosine similarity of their class prototypes does not improve on uniform averaging; the ablation without it wins on the majority of seeds.

Taxonomy refinement. Increasing the operation alphabet from 8 to 16
335 or 32 types (Proposition 1 guarantees these are strict refinements of the same labelling) does not produce a corresponding gain. The likely reason is visible in the class distribution: the memory-operation class that motivates the taxonomy fires on under 1% of AST nodes, so subdividing it cannot move an aggregate metric.

340 5. Discussion

5.1. *Why the abstraction transfers*

The result is consistent with a simple account. A lexical model’s features are drawn from the vocabulary of the projects it trained on; at test time on an unseen repository, much of its input is out-of-vocabulary, and what remains
345 correlates with project identity rather than with the weakness. The abstracted model’s alphabet is fixed by the C grammar and the operation taxonomy, so its input distribution is, by construction, the same on every project. The stability we observe supports this: fold-to-fold AUC deviation is roughly half that of the lexical model, which is what one expects if lexical variance is driven by
350 accidental vocabulary overlap between train and test.

This also explains why the effect is larger on ranking (AUC) than at the top of the ranking (AUPRC). The abstraction supplies a consistent notion of *what kind of operation* a node performs, which is enough to order functions by risk; it discards the specific API, which is often what distinguishes a genuinely
355 dangerous call from a benign one of the same type.

5.2. *Validating an abstraction is not optional*

The most transferable methodological point concerns RQ1. Program abstractions are routinely introduced as preprocessing and never evaluated: a taxonomy is listed, a mapping asserted, and downstream accuracy is reported.
360 Our experience argues this is unsafe. Two defects in our own mapping — substring matching that classified writes and predicates as memory operations, and a formatter class that absorbed 655 stream-I/O call sites — were invisible to every downstream metric and were caught only by annotating call sites and computing per-class precision. Both would have inflated the very statistic we
365 use to motivate the taxonomy.

The cost is modest: a few hundred annotated names. We would encourage the practice as a default for work that introduces a code abstraction, in the same way that a static analysis is expected to report precision and recall.

5.3. Negative results

370 We implemented a learned edge-weighting module and an affinity-weighted federated aggregation, and expected both to help. Neither improves on its ablation. We report them because the alternative — keeping components a paper names but its data does not support — is how unreproducible results enter the literature, and because the negative result is itself informative: it locates
375 the effect in the representation rather than in the machinery built around it.

The taxonomy-refinement result has a concrete explanation worth recording. The memory-operation class fires on under 1% of AST nodes, so subdividing it into allocation, release, copy and set cannot shift an aggregate metric however well-motivated the distinction is semantically. A finer taxonomy would need to
380 be paired with an evaluation that weights the nodes it refines.

5.4. Limitations

Graph fidelity. Our graphs are tree-sitter syntax trees with sibling and def-use approximation edges, not Code Property Graphs: interprocedural control flow and points-to-precise dependence are absent. Every system we compare
385 consumes the same graphs, so the comparison is internally fair, but the absolute ceiling is bounded by this choice.

Scale and language. Experiments use 8,000 functions per corpus, compact models, and C only. The abstraction is specified over a tree-sitter grammar and is therefore instantiable for other languages, but we have not demonstrated that,
390 and a taxonomy validated on C should not be assumed to transfer to, say, Rust’s ownership discipline.

Sampling and concentration. Corpus samples are taken in file order rather than at random, and project mass is concentrated (Chrome and `linux` supply about two thirds of BigVul). We hold those projects train-only and
395 report it, but a differently constructed sample could yield different absolute numbers.

Label noise. We inherit the labels of the underlying corpora, whose accuracy is itself contested in the literature that produced them.

Effect size. The AUPRC improvement is small and its interval’s lower
400 bound is at zero. The claim we defend is that the representation matters more
than the architecture choices it is usually compared against — not that cross-
project detection is solved.

5.5. A corollary for federated deployment

Because statistics computed over the abstracted graphs contain no project-
405 specific vocabulary, class-conditioned summaries can be shared between organi-
sations without transmitting code, and perturbed to satisfy differential privacy
at low dimensionality. We report this as a corollary rather than a contribu-
tion: in our experiments, ordinary parameter averaging without any privacy
constraint remained the stronger aggregation strategy, and we found no ev-
410 idence that the prototype-based alternative beats it absent a formal privacy
requirement.

6. Conclusion

We asked how much of the cross-project gap in vulnerability detection is
attributable to the program representation rather than the model, and answered
415 it by specifying an abstraction and measuring it under controls.

ϕ is a total labelling from a C grammar’s node alphabet onto an opera-
tion taxonomy, with a projection lattice that makes its granularities nested by
construction. Evaluated as a static analysis it exposes failure modes that down-
stream accuracy hides; evaluated as a representation, with graph, backbone,
420 depth, readout and training budget all held fixed, it improves cross-project
ranking by +0.064 AUC ($p = 0.001$) and +0.022 AUPRC ($p = 0.047$), winning
8 of 10 project-level folds, and roughly halves fold-to-fold variance.

We are equally clear about what we did not find. A learned edge-weighting
module, an affinity-weighted federated aggregation, and taxonomy refinement
425 beyond eight types all fail to improve on their ablations. Absolute performance
stays modest — an AUPRC lift of about $1.9\times$ over base rate — and cross-project
detection remains open.

The methodological suggestion we would most like to see adopted is the cheapest one: when a paper introduces a code abstraction, evaluate it as an analysis. A few hundred annotated call sites caught two defects in our own mapping that no downstream metric revealed, and that would otherwise have inflated the statistic used to motivate the design.

References

- [1] S. Chakraborty, R. Krishna, Y. Ding, B. Ray, Deep learning based vulnerability detection: Are we there yet?, *IEEE Transactions on Software Engineering* 48 (9) (2022) 3280–3296. doi:10.1109/TSE.2021.3087402.
- [2] P. Chakraborty, K. K. Arumugam, M. Alfadel, M. Nagappan, S. McIntosh, Revisiting the performance of deep learning-based vulnerability detection on realistic datasets, *IEEE Transactions on Software Engineering* 50 (8) (2024) 2163–2177, arXiv:2407.03093. doi:10.1109/TSE.2024.3423712.
- [3] Y. Ding, Y. Fu, O. Ibrahim, C. Sitawarin, X. Chen, B. Alomair, D. Wagner, B. Ray, Y. Chen, Vulnerability detection with code language models: How far are we?, in: *Proceedings of the 47th IEEE/ACM International Conference on Software Engineering (ICSE)*, IEEE, 2025, pp. 469–481, arXiv:2403.18624. doi:10.1109/ICSE55347.2025.00038.
- [4] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, Y. Zhong, VulDeePecker: A deep learning-based system for vulnerability detection, in: *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, Internet Society, 2018. doi:10.14722/ndss.2018.23158.
- [5] Z. Li, D. Zou, S. Xu, H. Jin, Y. Zhu, Z. Chen, SySeVR: A framework for using deep learning to detect software vulnerabilities, *IEEE Transactions on Dependable and Secure Computing* 19 (4) (2022) 2244–2258. doi:10.1109/TDSC.2021.3051427.
- [6] F. Yamaguchi, N. Golde, D. Arp, K. Rieck, Modeling and discovering vulnerabilities with code property graphs, in: *Proceedings of the IEEE*

Symposium on Security and Privacy (S&P), IEEE, 2014, pp. 590–604.
doi:10.1109/SP.2014.44.

- [7] Y. Zhou, S. Liu, J. K. Siow, X. Du, Y. Liu, Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 32, 2019, arXiv:1909.03496.
- [8] X. Cheng, H. Wang, J. Hua, G. Xu, Y. Sui, DeepWukong: Statically detecting software vulnerabilities using deep graph neural networks, ACM Transactions on Software Engineering and Methodology (TOSEM) 30 (3) (2021) 1–33. doi:10.1145/3436877.
- [9] Y. Li, S. Wang, T. N. Nguyen, Vulnerability detection with fine-grained interpretations, in: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), ACM, 2021, pp. 292–303. doi:10.1145/3468264.3468597.
- [10] R. Liu, Y. Wang, H. Xu, J. Sun, F. Zhang, P. Li, Z. Guo, VulLMGNNs: Fusing language models and online-distilled graph neural networks for code vulnerability detection, Information Fusion 115 (2025) 102748, arXiv:2404.14719. doi:10.1016/j.inffus.2024.102748.
- [11] Y. Luo, W. Xu, D. Xu, Detecting code vulnerabilities with heterogeneous GNN training, International Journal of Information Security 24, arXiv:2502.16835 (2025). doi:10.1007/s10207-025-01132-x.
- [12] M. Fu, C. Tantithamthavorn, LineVul: A transformer-based line-level vulnerability prediction, in: Proceedings of the 19th International Conference on Mining Software Repositories (MSR), ACM, 2022, pp. 608–620. doi:10.1145/3524842.3527927.
- [13] A. Lekssays, H. Mouhcine, K. Tran, T. Yu, I. Khalil, LLMxCPG: Context-

aware vulnerability detection through code property graph-guided large language models, arXiv preprintArXiv:2507.16585 (2025).

- 485 [14] F. Weissberg, L. Pirch, E. Imgrund, J. Möller, T. Eisenhofer, K. Rieck, LLM-based vulnerability discovery through the lens of code metrics, in: Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE), ACM, 2026, arXiv:2509.19117. doi:10.1145/3744916.3764574.
- 490 [15] R. Widyasari, M. Weyssow, I. C. Irsan, H. W. Ang, F. Liauw, E. L. Ouh, L. K. Shar, H. J. Kang, D. Lo, Let the trial begin: A mock-court approach to vulnerability detection using LLM-based agents, in: Proceedings of the 48th IEEE/ACM International Conference on Software Engineering (ICSE), ACM, 2026, arXiv:2505.10961. doi:10.1145/3744916.3773256.
- 495 [16] H. Yamamoto, D. Wang, G. K. Rajbahadur, M. Kondo, Y. Kamei, N. Ubayashi, Towards privacy preserving cross project defect prediction with federated learning, in: Proceedings of the 30th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), IEEE, 2023, pp. 485–496. doi:10.1109/SANER56733.2023.00052.
- 500 [17] S. Liu, G. Lin, L. Qu, J. Zhang, O. D. Vel, P. Montague, Y. Xiang, CD-VulD: Cross-domain vulnerability discovery based on deep domain adaptation, IEEE Transactions on Dependable and Secure Computing 19 (1) (2022) 438–451. doi:10.1109/TDSC.2020.2984505.
- [18] X. Li, Y. Xin, H. Zhu, Y. Yang, Y. Chen, Cross-domain vulnerability detection using graph embedding and domain adaptation, Computers & Security 125 (2023) 103017. doi:10.1016/j.cose.2022.103017.
- 505 [19] C. Zhang, B. Liu, Y. Xin, L. Yao, CPVD: Cross project vulnerability detection based on graph attention network and domain adaptation, IEEE Transactions on Software Engineering 49 (8) (2023) 4152–4168. doi:10.1109/TSE.2023.3285910.
- 510

- [20] Z. Cai, Y. Cai, X. Chen, G. Lu, W. Pei, J. Zhao, CSVD-TF: Cross-project software vulnerability detection with TrAdaBoost by fusing expert metrics and semantic metrics, *Journal of Systems and Software* 213 (2024) 112038. doi:10.1016/j.jss.2024.112038.
- 515 [21] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, B. A. y Arcas, Communication-efficient learning of deep networks from decentralized data, in: *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, Vol. 54 of *Proceedings of Machine Learning Research*, PMLR, 2017, pp. 1273–1282.
- 520 [22] S. Shanbhag, S. Chimalakonda, Exploring the under-explored terrain of non-open-source data for software engineering through the lens of federated learning, in: *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), Ideas, Visions and Reflections*, ACM, 2022, pp. 1610–1614. doi:10.1145/3540250.3560883.
- 525 [23] Y. Yang, X. Hu, Z. Gao, J. Chen, C. Ni, X. Xia, D. Lo, Federated learning for software engineering: A case study of code clone detection and defect prediction, *IEEE Transactions on Software Engineering* 50 (2) (2024) 296–321. doi:10.1109/TSE.2023.3347898.
- 530 [24] P. Zhou, M. Hu, X. Xie, Y. Liu, M. Chen, VulFL: An empirical study of vulnerability detection using federated learning, *arXiv preprint-ArXiv:2411.16099* (2024).
- 535 [25] C. He, K. Balasubramanian, E. Ceyani, C. Yang, H. Xie, L. Sun, L. He, L. Yang, P. S. Yu, Y. Rong, P. Zhao, J. Huang, M. Annavaram, S. Avestimehr, FedGraphNN: A federated learning system and benchmark for graph neural networks, in: *Proceedings of the ICLR Workshop on Distributed and Private Machine Learning (DPML)*, 2021, arXiv:2104.07145.
- [26] C. Zhang, T. Yu, B. Liu, Y. Xin, Vulnerability detection based on federated

- learning, *Information and Software Technology* 167 (2024) 107371. doi:
540 10.1016/j.infsof.2023.107371.
- [27] L. Wang, J. Liu, X. Gao, et al., FedDense: Tackling non-IID graphs via decoupled structure and feature in federated graph learning, in: *Database Systems for Advanced Applications (DASFAA)*, LNCS 15988, Springer, 2026. doi:10.1007/978-981-95-3906-2_22.
- 545 [28] M. Abadi, A. Chu, I. Goodfellow, H. B. McMahan, I. Mironov, K. Talwar, L. Zhang, Deep learning with differential privacy, in: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, ACM, 2016, pp. 308–318. doi:10.1145/2976749.2978318.
- 550 [29] K. Bonawitz, V. Ivanov, B. Kreuter, A. Marcedone, H. B. McMahan, S. Patel, D. Ramage, A. Segal, K. Seth, Practical secure aggregation for privacy-preserving machine learning, in: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, ACM, 2017, pp. 1175–1191. doi:10.1145/3133956.3133982.
- 555 [30] J. Fan, Y. Li, S. Wang, T. N. Nguyen, A C/C++ code vulnerability dataset with code changes and CVE summaries, in: *Proceedings of the 17th International Conference on Mining Software Repositories (MSR)*, ACM, 2020, pp. 508–512. doi:10.1145/3379597.3387501.
- 560 [31] Y. Chen, Z. Ding, L. Alowain, X. Chen, D. Wagner, DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection, in: *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, ACM, 2023, pp. 654–668. doi:10.1145/3607199.3607242.